Five Bits per Cell:
What a 4-Bit KV Cache Needs to Match `int8`

A falsification plan for the exact-cell window's successor

Matthias Kronenberger

Version 0.1 · built 2026-10-25, 09:51 CET · first version 2026-10-25

Abstract

The exact-cell window stores a 4-bit KV cache at 5.29 bit/cell (1,016 MiB at 128K, 36% below `int8`'s 1,584 MiB), but `int8` remains ahead on the statistics that carry weight: the paired 4K mean, the 128K whole segment, p99 and the last 1,024 positions on both texts. The predecessor paper left two bars for a follow-up: opt-in parity at 6.0 bit/cell, and a default change at any rate below 8.25. Neither is a product target — 6.0 sits above the shipped 5.29, and a win at 8.2 saves 0.6% of the cache. This paper asks the follow-up question in product terms: what would a point have to look like that stores at most **5.0 bit/cell** — 6% below the shipped window and 39% below `int8` — and is not behind `int8` on any robust statistic? We fix the target operationally (six criteria, set before the first run), compute the byte budget (0.50 bit/cell for the ring: 4,096 cells in fp16, 7,943 with an `int8` ring, 5,461 at 12 bit, against 6,464 today), locate the remaining error from the predecessor's measurements (the bulk, not the overlay; and immediately outside the window, not only far from it), list six levers with their bit cost and their kill, and give a five-stage falsification ladder with explicit stop conditions. *No number in this paper is a measurement of the 5.0-bit point.* The measured numbers are the predecessor's; the 5.0 numbers are byte-model arithmetic on them and are labelled as such.

1 Introduction

The predecessor paper ended with a measured trade-off and an open question [1]. The exact-cell window keeps the most recent S=256 cells and R=64 sink cells of each of the 12 cache-carrying QSA layers in fp16 while the rest of the cache stays in the 4-bit Lloyd format. At 128K it uses 5.29 bit/cell and 1,016 MiB against `int8`'s 8.25 bit/cell and 1,584 MiB, frees 568 MiB for expert weights (416 more resident slots), holds prefill throughput (528 against 526 t/s) and keeps decode inside run-to-run spread. On the fixed cwe corpora it removes 73% of the plain 4-bit base's KL gap to `int8` at 4K and 34–58% of its 128K tail-mean KL. It does not beat `int8`: at 4K `int8` is ahead on the paired mean ($t=-3.55$, 7 of 8 segments), and at 128K it keeps the p99, the last 1,024 positions and the whole segment on both texts.

That paper named two bars for a successor [1]. Bar A asked for opt-in parity within half of the measured gaps at no more than 6.0 bit/cell. Bar B asked for a default change: not behind `int8` on the paired 4K mean, the 128K whole segment or p99 on either text, at any rate below 8.25 bit/cell. Both were written to be honest about what the evidence supports, and both are too weak as a *target*.

Bar A's ceiling sits above today's measured point: an arm at 6.0 bit/cell would be 13% *larger* than the shipped window while merely tying `int8` within a margin. Bar B has no byte ceiling at all, so a win at 8.2 bit/cell saves 0.6% of the cache — a quality result without a product. A target that is worth a branch has to bind both sides at once: a byte ceiling below today's point and a quality floor at `int8`.

This paper sets that target at **5.0 bit/cell**: 6% below the shipped window (5.29) and 39% below `int8` (8.25), with `int8` not ahead on any robust statistic. Five bits is not a round number borrowed from the literature; it is the point where the arithmetic below still leaves a ring large enough to cover the cells the indexer reads, while the bulk has to become better than 4-bit Lloyd to close the rest. The paper's contributions are:

- an operational definition of "better than `int8` in all areas" (six criteria, fixed before the first run, with the statistics that are deliberately excluded and why);
- the byte budget at 5.0 bit/cell, including three ring tiers (fp16, `int8`, 12 bit) and the break-even move;
- the location of the remaining error, read off the predecessor's measurements: the bulk carries it, and the last-1,024 loss sits immediately outside the exact window, not only far from it;
- six levers with their bit cost, their gate and their kill, and a five-stage ladder with explicit stop conditions.

What this paper is not: it reports no new measurement, changes no engine code, and does not claim that 5.0 bit/cell is reachable. Its 5.0 numbers are byte-model arithmetic on two measured points, and every such number is marked. The paper is a pre-registration: it fixes the target, the byte ceiling and the kills before the first GPU run, so that a later result cannot be fitted to a moving bar. A negative result — the ladder ending without a winner — is an intended outcome and is cheaper than an open-ended sweep.

2 The Measured Point and the Target

2.1 What the predecessor measured

Table 1 collects the numbers this paper builds on. All of them are the predecessor's [1, 19]; none was re-run here. The engine is 0.1.44.1 for the quality and runtime tables and 0.1.47 for the needle pair, on one RX 9060 XT (gfx1200),

Windows, ROCm/HIP 7.1, Swift 1.5 IQ3_XXS, `--spec 4`. The KL is teacher-forced against an fp16 KV reference and computed over the reference's top-64 tokens, renormalized.

Two readings of Table 1 shape everything that follows. First, the window's deficit is not uniform: it is a whole-segment and tail-quantile deficit, while the tail *mean* favors each arm on one text and reverses sign between texts. Second, the deficit survives even where 256 of the last 1,024 positions are held exactly (the window's S=256): the remaining 768 positions are 4-bit, and `int8` is ahead on that statistic on both texts. A larger exact overlay is the obvious response, and the next section shows why it is not available at five bits.

2.2 The target, fixed before the first run

The target has six criteria. All are checked in the order given; a criterion that fails ends the arm. The statistics are the predecessor's, so the comparison is paired and same-harness.

Three choices in Table 2 need a reason. The tail mean is excluded because its sign is not stable across the two 128K texts (Table 1): a statistic that favors each arm once cannot carry a target, and the predecessor already used the p99, the last 1,024 and the whole segment for its robust reading. cwe-2k is excluded because the window is already statistically indistinguishable from `int8` there ($t = -1.92$, window ahead in 2 of 8 segments); it would be a criterion no arm can fail, not a test. Equality is accepted on every statistic because the byte rate is 39% lower: at five bits a tie is a product, while at 8.25 it is not.

K5's 10% is the only threshold without a measured basis; it is a choice, set because the predecessor's window is at +0.4% and because the one measured small-ring configuration (B=512) pays 80% of prefill. The threshold exists so that a quality win cannot be bought with an unmeasured prefill collapse. K6 keeps the predecessor's gates and adds the two open review gates, because a default claim needs the full cell-to-row parity and the other two backends.

Finally, what "all areas" cannot mean here. The harness measures single-segment, teacher-forced KL on one task family, on one GPU, one operating system, one weight set. It does not measure multi-turn growth, concurrency, mid-segment KL against the other arms, or a second task family. The target is therefore a claim about the statistics in Table 2 and nothing wider; Section 8 states the bound.

3 The Byte Budget at Five Bits

The predecessor's byte model is one line [1]:

$$\text{bits/cell} \;\approx\; 4.50 \;+\; \frac{16\,(R+S+B)}{L}, \qquad (1)$$

where L is the context length in cells, the 4.50 is the 4-bit base (1,024 values per cell and layer in 32 blocks of 18 bytes = 576 B), and the second term is the resident ring: the exact side copy costs 16 bit/cell *in addition* to the 4-bit row it duplicates, which is why the measured point is $4.50 + 16 \times 6{,}464/131{,}072 = 5.289$.

The equation is the whole argument for the ceiling. At 5.0 bit/cell the ring may spend 0.50 bit/cell, which at L=131,072 is 4,096 cells in fp16. With S+R=320 that leaves $B \le 3{,}776$, against 6,144 today: the fp16 ring has to shrink by 37% (6,464 to 4,096 cells) while the quality floor rises. The ring does not have to be fp16, though. An `int8` side copy costs 8.25 bit/cell (the default's 8 bits plus its 0.25-bit scale share), which buys 7,943 cells — 23% more than today — and a 12-bit ring buys 5,461. Table 3 lists the three tiers.

The tier choice is the paper's central hypothesis, and it is not a byte trick. `int8` is the quality reference, not fp16: a ring that holds its cells at `int8` precision makes the ring region *equal* to `int8` by construction, and spends the remaining quality budget on the bulk. If the measured deficit is concentrated in the ring region — which the last-1,024 statistic suggests (Section 4) — then an `int8` ring attacks exactly the cells that carry the loss, at 8.25 rather than 16 bits. The price is that the ring is no longer exact: a cell that leaves the ring is quantized, and the fault rule of the A-series (a promised exact cell that is no longer present) has to be restated for a tiered ring (Section 5).

Two consequences follow from Table 3. The break-even against `int8` moves from about 27,600 cells (6,464-ring) to about 17,500 (4,096-ring), so a 5.0 point is cheaper than `int8` from roughly 17.5K cells upward — at 24K it is already cheaper, and setup's 32K offer stays valid and gains margin. And the product arithmetic: linear between the two measured points, 5.0 bit/cell is about 960 MiB at 128K — 624 MiB (39%) below `int8` and 56 MiB (6%) below today's window — and about 457 more expert slots than `int8` at 416 slots per 568 MiB. The slot figure is model- and quantization-dependent (Swift 1.5 IQ3_XXS, RX 9060 XT); it is arithmetic, not a measurement.

The gain is memory and slots, not speed. The predecessor's window is at prefill parity, and the smaller ring could cost prefill: the only measured small-ring arm (B=512) ran at 105 against 528 t/s. That is why K5 exists, and why a 5.0 arm that wins on quality but loses more than 10% of prefill is dead in this plan.

One more term belongs in the budget: a per-channel scale before the rotation (lever L3) costs one fp16 value per 128 cells on the K half, $16/128 \times 0.5 \approx 0.06$ bit/cell. At 5.0 that leaves 0.44 bit/cell for the ring — 3,604 fp16 cells or 6,990 `int8` cells — so the levers compete for the same 0.50 and cannot be priced independently.

4 Where the Error Sits

The predecessor's measurements answer, partially, where the remaining error lives. Four facts matter for the plan; none is a new measurement.

A larger fp16 ring is arithmetically closed. Under a 6.0 ceiling, 1.5 bit/cell of headroom buys at most 9.4% of the cells (1.5/16, about 12,300 cells at 128K); at 5.0 it buys 3.1% (4,096). The measured 512/128 arm doubles the exact overlay at 4K (320 to 640 positions) and ties 256/64 ($t = +0.02$) while

Table 1: The measured point this paper starts from. `int8` is the default and the quality reference; the window is `--kv q4lm8v` with `STRATA_KV_WINDOW=1`, S=256, R=64, automatic chunk B=6,144. KL values are mean KL against the fp16 KV reference (top-64, renormalized); lower is better. Δ is window minus `int8`, so a positive value is a window loss. T1/T2 are the two 128K texts. Source: the predecessor's Tables 4, 6 and 7 and its Section 7.3 [1, 19].

Statistic	`int8`	window 256/64	Δ
Rate at 128K (bit/cell)	8.25	5.29	−2.96
KV memory at 128K (MiB)	1,584	1,016	−568 (−36%)
cwe-2k, mean KL	0.00503	0.00702	+0.00199 ($t = -1.92$)
cwe-4k, mean KL	0.01688	0.02090	+0.00402 ($t = -3.55$; 7/8)
128K T1, whole segment	0.10715	0.13312	+0.02597
128K T1, p99	0.01655	0.02977	+0.01322
128K T1, last 1,024	0.00265	0.00444	+0.00179
128K T1, tail mean (4,096)	0.00959	0.00663	−0.00296 ($t = -0.74$)
128K T2, whole segment	0.06435	0.07524	+0.01089
128K T2, p99	0.0165	0.0210	+0.0045
128K T2, tail mean (4,096)	0.00324	0.00437	+0.00113
Prefill (t/s)	526	528	+0.4%
Decode (t/s)	—	−1.0	inside spread
Gates	SHA, 12/12, 6/6+6/6	same	green

Table 2: The 5.0-bit target. "Not behind" means the arm's value is less than or equal to `int8`'s; a tie is not a loss. Anchors are the measured window values from Table 1 and are shown for scale, not as thresholds. K1–K3 are the quality floor, K4 the byte ceiling, K5 the runtime bound, K6 the correctness gates. C1 (full cell-to-row parity) and C2 (CUDA/SYCL builds) are the two review gates still open in the predecessor.

#	Criterion	Anchor (window today)
K1	cwe-4k paired mean KL (arm − `int8`) ≤ 0, upper 95% bound < +0.001 nats, and `int8` ahead in at most 4 of 8 segments	+0.00402, $t = -3.55$, 7/8
K2	cwe-128k whole-segment KL ≤ `int8` on *both* texts	+0.02597 (T1), +0.01089 (T2)
K3	cwe-128k p99 *and* last 1,024 ≤ `int8` on both texts	p99 +0.01322/+0.00450; last 1,024: 0.00444 vs 0.00265
K4	effective rate at 128K ≤ 5.0 bit/cell (byte model written down before the run)	5.29, 39% below `int8`
K5	prefill at most 10% below `int8` (526 t/s); decode inside run-to-run spread	528 vs 526 t/s; B=512 costs −80%
K6	`int8` dump-logits SHA unchanged; parity 12/12; needles 6/6 + 6/6; setup suite; plus C1 and C2	all green today except C1/C2

costing 7 MiB more at 128K and a slower decode. Spending the ceiling on a bigger fp16 ring is therefore not a route: the bytes are not there, and the one measured doubling bought nothing.

The loss sits next to the window, not only far from it. The last-1,024 statistic is the sharpest evidence. Of those 1,024 positions, 256 are inside the window and stored exact (read exact when the indexer selects them); the other 768 are 4-bit. `int8` wins that statistic on both texts (0.00265 against 0.00444 on T1). So the deficit is produced by cells *immediately* outside the exact overlay — cells that a tiered ring of the same size would cover better, because an `int8` ring cell is closer to `int8` than a 4-bit cell is. This is a static reading of the design plus a measured statistic, not a measurement of the tiered ring.

The tail mean cannot carry a target. It favors the window on T1 (0.00663 against 0.00959, $t = -0.74$) and `int8` on T2 (0.00324 against 0.00437). The sign flips between two texts from the same generator, so the target uses the p99, the last 1,024 and the whole segment instead. A 5.0 arm that wins the tail mean on one text has won nothing.

2K is saturated. The window is already indistinguishable from `int8` at 2K ($t = -1.92$, window ahead in 2 of 8 segments). A target statistic that no arm can fail is not a test; 2K is reported, not used.

Together these facts point one way: the bulk must become a better code than 4-bit Lloyd, and the ring must become cheaper per protected cell. Neither alone reaches 5.0: a smaller fp16 ring loses the statistics `int8` wins, and a better bulk without the byte side stays above the ceiling.

The predecessor also closed five doors, and this plan does not reopen them [22]: clipping after the Walsh–Hadamard (worse 2K KL), Hadamard on/off as a quality arm (a tie on the 16-level base at 2K and 4K), the K-versus-V bit contrast ($t = +0.05$), a 3-bit base without the window (worse), and R=8 as a sink count (too small). The published direction that keys keep the bits [14] stands as its authors' result on their models; it is not a reason to rerun a contrast this model has already failed.

5 Six Levers

Table 4 lists the levers in the order the ladder tests them. Each has a bit cost that comes out of the same 0.50, a gate that must pass before it is built, and a kill. None is a result.

Table 3: The ring budget at 5.0 bit/cell (L=131,072, S+R=320). The side copy is additional to the 4-bit row. The 12-bit row is priced at 12.0 bit/cell; with `int8`'s 0.25-bit scale share it would be 5,349 cells. The two lower tiers are arithmetic, not formats: neither exists in the engine yet.

Ring tier	bit/cell	R+S+B	B	vs. today
fp16 (today)	16.0	4,096	3,776	−37%
`int8`	8.25	7,943	7,623	+23%
12 bit	12.0	5,461	5,141	−15%
today's point	16.0	6,464	6,144	—

Table 4: The six levers. "Cost" is the side-copy budget each one consumes at 5.0 bit/cell; the base's 4.50 is fixed. A lever with no cost changes only the code inside the existing 4-bit block. Gates refer to the ladder stages of Table 5.

Lever	What changes	Cost	Kill
L1 level layout	16 levels on K and V instead of 16/8, same 4-bit block (A2)	0	tie at 4K
L2 level table	a different 16-level table for the same block (A5)	0	no K1 at 4K
L3 per-channel K	one scale per 128 K cells, before the Walsh–Hadamard (A1)	0.06	no heavier pre-rotation tail (S2)
L4 ring tier	`int8` or 12-bit side copy instead of fp16; geometry from the knee sweep (B1)	−7.75 or −4.0 vs. fp16	no K1–K5 in any tier
L5 selection-following pool	exact bytes on cells the indexer returned, every cell still addressable at 4 bit (E3)	0 (same 16/cell)	fewer than half of selections in R+S (S1)
L6 residual correction	a low-rank or sparse residual on the 4-bit bulk (GEAR form)	residual bits	dense residual (E4)

L1 and L2: the code inside the block. The 4-bit block holds K at 16 levels and V at 8. Giving V 16 levels costs no bytes and is the cheapest quality experiment in the plan; a tie kills it, and a tie is the likely outcome given that the predecessor's K-versus-V contrast did not support an asymmetry. A different 16-level table (L2) is the same experiment with a different codebook and the same kill. Neither touches the byte budget, which is why they come first: if the bulk cannot be improved inside 4 bits, the plan's remaining levers all cost bytes.

L3: the channel axis before the rotation. The writer quantizes after the Walsh–Hadamard, which mixes the 64 rotated with the 192 unrotated dimensions. KIVI's per-channel key quantization assumes channels survive to the quantizer [4]; here they do not, unless the scale is applied before the transform. The lever is a per-channel scale on the K half before the transform, at 0.06 bit/cell. Its gate is a measurement, not an argument: the second moments of the K channels before and after the transform, split into the two slices. If the pre-rotation channels are not the heavier tail, the lever is not built. Per-channel scales after the transform are not this lever — the transform has already mixed the channels.

L4: the ring tier. This is the byte lever, and the one the paper's hypothesis rests on. Three tiers are priced in Table 3; the ladder measures them as three byte variants of the same bulk winner. The `int8` tier has an engineering argument beyond its arithmetic: the engine already writes and reads `int8` KV for the default, so the tier reuses a code path that is measured and gated, while a 12-bit tier is a new format with its own writer, reader and parity suite. Against that stands the fault rule: today's window promises exact cells, and a tiered ring promises *near-exact* cells. The rule has to be restated — a cell that leaves the ring is quantized to the bulk, never silently dropped — and the parity test (C1) has to cover the tier boundary. The knee sweep (B1) belongs here too: it can redistribute S and R, but it cannot create bytes, so it is a refinement of the tier, not a way around the ceiling.

L5: spend the exact bytes where the indexer looks. The predecessor's counter question is whether the positional overlay covers the cells the indexer actually selects. If fewer than half of the selections fall in R+S, the positional overlay is in the wrong place, and the alternative is a pool that follows the selection: every cell stays addressable at 4 bits, and the higher-precision copy goes to cells the indexer returned. The selected set changes per query, so the pool is a union or the last selection, and a promised cell that has left the pool is the same fault as a dropped cell. This lever does not start until the counter says the overlay is misplaced; if the counter says the overlay is right, L5 is dead and L4 carries the plan.

L6: the residual. A low-rank or sparse residual on top of the 4-bit bulk is the GEAR form [6]. It is last because its cost is not known before the residual's shape is measured: a dense residual costs bits everywhere, a sparse one costs a mask plus a few values. The gate is therefore the residual's shape on the existing 4K pair — dense kills the lever, sparse prices it. L6 is the only lever that can improve cells the ring does not cover without changing the ring, which is why it stays on the list even though it is the least specified.

6 The Ladder

The ladder has five stages. Each stage asks one question, and the next stage starts only if the previous stage's kill did not fire. The risk the ladder manages is not that an arm fails — failure is the intended outcome — but that the engine's default or the `int8` SHA moves while the plan is being tested. Every stage is opt-in, and every stage writes its byte model down before it runs.

The stop conditions are part of the plan, not an afterthought:

- S0 flips the ranking or the sign count: the bar is undefined; no engine work until the 128K reading exists under the same definition.
- S3 finds no arm reaching K1 at the weaker 6.0 bar: the bulk is not the lever, and the branch ends with a documented negative result. No 128K run is started.
- S4's three variants all miss K1–K5: 5.0 is not reachable with these six levers. The branch either ships the 6.0 opt-in fallback (parity within half of today's gaps at $\leq$ 6.0 bit/cell) or reports the negative result; it does not start a sweep.
- A variant loses more than 10% of prefill: dead, regardless of quality.
- Any stage moves the `int8` SHA or fails parity: stop and fix, no quality claim.

The budget is deliberately small: at most three 4K arms in S3, one 128K block of three variants by two texts in S4, and no 128K sweep. If S3 kills, the whole plan costs one counter run, one debug statistic and three 4K arms — less than a single 128K sweep.

7 Relation to Prior Work

The levers are not new mechanisms; they are the mechanisms that survive this model's geometry, and each maps to a published line of work. The mapping is the honest form of a related-work section for a plan: it says which paper each lever borrows from, and what that paper measured on its own models, not on this one.

Per-channel key scales are KIVI's axis [4]; KIVI pairs them with per-token value scales and a 2-bit budget plus an fp16 residual window, and reports negligible accuracy loss on its models. Here the rotation sits between the channels and the quantizer, which is why L3 is gated on a moment measurement rather than ported. KVQuant's per-channel pre-RoPE treatment and sparse outliers [5] are the same axis in a different placement; this cache rotates 64 of 256 dimensions and normalizes after RoPE, so a pre-RoPE port would quantize a different tensor. GEAR's residual [6] is L6; its near-lossless claim rests on a low-rank plus sparse residual, and the sparse part is the part E4 would have to find here.

The exact overlay itself is the literature's sliding-window and sink protection: StreamingLLM's sinks [3], SKVQ's sliding window [7], IntactKV's pivot tokens [8], RotateKV's sink handling [9]. The predecessor implements that mechanism on the cells this model reads; L4 changes the precision of the protected cells, not their selection. The selection-following variant (L5) is the token/precision trade-off of the top-k line [16], with the indexer's selection left untouched because replacing it would change the model. Layer-wise mixed precision [15] is not a lever here: only 12 of 48 layers carry this cache, and no per-layer error signal exists yet. Learned rotations and rewritten projections [10, 11, 12] are a different model, not a cache flag. The system-aware 4-bit formats [13] are the same family as Lloyd plus Walsh–Hadamard; the predecessor's on/off ablation was a tie at 2K and 4K, so L1/L2 test the code, not the transform.

Two deployed anchors bound the claim. `llama.cpp`'s `q4_0`/`q8_0` KV types [17] are the field's version of the same trade-off, measured on document perplexity, not on this cwe protocol; the predecessor's `q4_0` anchor is a different harness and is not comparable. FP8 [18] is not an attention kernel on this GPU, which is why the two-times point here is `int8` and the frontier is 4 bit.

8 Threats to Validity

The first threat is the obvious one: there is no 5.0-bit measurement. Every 5.0 number in this paper is either the byte model of Equation 1 (a linear model validated at one measured point, 5.289) or a linear interpolation between the two measured points (8.25/1,584 and 5.29/1,016). The model ignores per-layer variation, block-scale overhead beyond the 0.25 bit/cell assumed for `int8`, and any non-linearity in the ring's cost; the 960 MiB and 457-slot figures inherit all three. The three ring tiers are arithmetic: the `int8` and 12-bit tiers do not exist in the engine, and their bit costs are the default's and a nominal 12.0.

The second threat is the reference. "Not behind `int8`" is not "not behind fp16": `int8` carries its own quantization error, and a 5.0 arm that matches it inherits that error. The target is deliberately relative, because the product question is whether to keep `int8` as the quality default, not whether 4-bit can be lossless. A reader who wants an fp16-relative claim will not find one here, and the predecessor's KL values are all against the fp16 reference for exactly that reason.

The third threat is coverage. The quality evidence is one task family (RULER complex word extraction), scored teacher-forced, single-segment, on one model, one GPU, one operating system, one stack and one weight set. The target's "all areas" is therefore all *measured* areas; multi-turn growth, concurrency, mid-segment KL against the other arms and a second task family are unmeasured, and the plan does not claim them. The 128K results are one text each with adjacent blocks, so their t values are descriptive; the 4K results have eight segments and are the only place where the sign count is a real statistic.

The fourth threat is the bar itself. K1's upper bound and K5's 10% are choices, not measurements; K4's 5.0 is a product decision. The plan mitigates this by fixing all of them before the first run and by reporting the raw numbers either way, so a later disagreement can be about the threshold rather than

Table 5: The ladder. S0 and S1 are offline or a counter; S3 costs 4K GPU time for at most three arms; S4 is the only 128K block (three byte variants, two texts). A stage that is killed ends the branch with a documented negative result rather than a wider sweep.

Stage	Question	Kill	Cost
S0	Is the bar itself stable? Rescore one existing 4K pair under a second KL definition (top-256 or full vocabulary) on the same positions, signs reported.	ranking or sign flips ⇒ no kernel until 128K is reread under the same definition; K1/K2 undefined	scorer only
S1	Do the indexer's selections fall in R+S? Count, on one 128K prefill, the share in R, in S, in neither, per QSA layer; plus the 4K residual's shape (E4).	$<$ half in R+S ⇒ the positional overlay is misplaced: L5 becomes the candidate and B1 does not run; dense residual ⇒ no L6	one counter run
S2	Is the channel axis heavier *before* the rotation? Second moments of the K channels before and after the transform, split into the 64 rotated and 192 unrotated dimensions.	no heavier pre-rotation tail ⇒ L3 not built	debug statistic, `int8` untouched
S3	Does the cheap bulk tool suffice? 4K, paired against `int8`: (a) 16/16, (b) alternative table, (c) L3 if S2 survived. Each arm reports K1, K4 and bit/cell.	no arm reaches K1 at $\leq$ 6.0 bit (the weaker bar, so not at 5.0 either) ⇒ the bulk is not the lever; no 128K, back to S1 or stop	$\leq$ 3 4K arms
S4	Which 5.0 composition holds? The S3 winner as bulk, three byte variants at 128K, both texts, all K statistics, byte model first: (a) fp16 ring, R+S+B=4,096; (b) `int8` ring, 7,943; (c) 12-bit ring, 5,461.	no variant meets K1–K5 ⇒ 5.0 is not reachable with these levers: fall back to the 6.0 opt-in bar or publish the negative result; prefill $<$ 90% of `int8` kills the variant even if quality passes	one 128K block

about the data. What it cannot mitigate is that a target set by the authors of the arm is a target the authors can move; the pre-registration and the published kills are the only defence, and they are worth exactly as much as the discipline of the run.

9 Conclusion

The window's successor has a precise job: store at most 5.0 bit/cell — 39% below `int8`, 6% below the shipped window — and not be behind `int8` on the paired 4K mean, the 128K whole segment, p99 or the last 1,024 positions on either text, with prefill within 10% and the correctness gates green. The byte budget says the ring may spend 0.50 bit/cell: 4,096 cells in fp16, 7,943 with an `int8` ring, 5,461 at 12 bit, against 6,464 today. The measured evidence says the remaining error is in the bulk and immediately outside the window, that a larger fp16 ring is arithmetically closed and was already tried at 4K without effect, and that the tail mean cannot carry a target. The six levers and the five-stage ladder turn that into a finite experiment: one counter run, one debug statistic, at most three 4K arms and one 128K block, with kills that end the branch instead of widening it.

The plan's most likely outcomes are a negative result and a 6.0-bit fallback, in that order. That is not a weakness of the plan; it is what a pre-registered target looks like before the data exist. If the `int8` ring plus a better bulk reaches 5.0, the window is not improved but replaced: the same 128K context would cost about 960 MiB instead of 1,584, with no statistic behind `int8`. If it does not, the negative result is the deliverable, and the shipped window stays the honest memory point at 5.29.

References

[1] M. Kronenberger. *Hardening the Exact-Cell KV Window for* `--kv q4lm8v`*: Design, Decisions, and a Measurement-First Validation.* In-repo, `docs/paper/KV-Window-Hardening.pdf`, version 1.6, 2026-10-25.

[2] M. Kronenberger. *Ansatz-Verbesserungen: KV-Cache-Quantisierung & Exact-Cell-Fenster* (working document, German). In-repo, `docs/paper/Ansatz_Improvements.md`, 2026-10-25.

[3] G. Xiao, Y. Tian, B. Chen, S. Han, M. Lewis. *Efficient Streaming Language Models with Attention Sinks.* arXiv:2309.17453, 2023 (ICLR 2024). Code: https://github.com/mit-han-lab/streaming-llm.

[4] Z. Liu, J. Yuan, H. Jin, S. Zhong, Z. Xu, V. Braverman, B. Chen, X. Hu. *KIVI: A Tuning-Free Asymmetric 2bit Quantization for KV Cache.* ICML 2024, arXiv:2402.02750. Code: https://github.com/jy-yuan/KIVI.

[5] C. Hooper, S. Kim, H. Mohammadzadeh, M. W. Mahoney, Y. S. Shao, K. Keutzer, A. Gholami. *KVQuant: Towards 10 Million Context Length LLM Inference with KV Cache Quantization.* NeurIPS 2024, arXiv:2401.18079. Code: https://github.com/SqueezeAILab/KVQuant.

[6] H. Kang et al. *GEAR: An Efficient KV Cache Compression Recipe for Near-Lossless Generative LLM Inference.* arXiv:2403.05527, 2024.

[7] H. Duanmu, Z. Yuan, X. Li, J. Duan, X. Zhang, D. Lin. *SKVQ: Sliding-window Key and Value Cache Quantization for Large Language Models.* arXiv:2405.06219, 2024.

[8] R. Liu, H. Bai, H. Lin, Y. Li, H. Gao, Z. Xu, L. Hou, J. Yao, C. Yuan. *IntactKV: Improving Large Language Model Quantization by Keeping Pivot Tokens Intact.* ACL Findings 2024, arXiv:2403.01241.

[9] Z. Su, Z. Chen, W. Shen, H. Wei, L. Li, H. Yu, K. Yuan. *RotateKV: Accurate and Robust 2-Bit KV Cache Quantization for LLMs via Outlier-Aware Adaptive Rotations.* arXiv:2501.16383, 2025 (IJCAI 2025).

[10] S. Ashkboos, A. Mohtashami, M. L. Croci, B. Li, P. Cameron, M. Jaggi, D. Alistarh, T. Hoefler, J. Hensman. *QuaRot: Outlier-Free 4-Bit Inference in Rotated LLMs.* arXiv:2404.00456, 2024.

[11] Z. Liu, C. Zhao, I. Fedorov, B. Soran, D. Choudhary, R. Krishnamoorthi, V. Chandra, Y. Tian, T. Blankevoort. *SpinQuant: LLM quantization with learned rotations.* ICLR 2025, arXiv:2405.16406.

[12] C.-C. Chang, W.-C. Lin, C.-Y. Lin, C.-Y. Chen, Y.-F. Hu, P.-S. Wang, N.-C. Huang, L. Ceze, M. S. Abdelfattah, K.-C. Wu. *Palu: Compressing KV-Cache with Low-Rank Projection.* arXiv:2407.21118, 2024.

[13] J. Jia, J. Li, Z. Zhou, J. H. Heo, J. Wang, T. Dao, S. L. Song, B. Athiwaratkun, C. Xu, T. Zhang, X. Wu. *SAW-INT4: System-Aware 4-Bit KV-Cache Quantization for Real-World LLM Serving.* arXiv:2604.19157, 2026.

[14] M. Hariri, A. Luo, W. Chen, S. Zhong, T. Zhang, Q. Wang, X. Hu, X. Han, V. Chaudhary. *Quantize What Counts: More for Keys, Less for Values*. ACL 2026, arXiv:2502.15075.

[15] X. Li, Z. Xing, Y. Li, L. Qu, H. Zhen, Y. Yao, W. Liu, S. J. Pan, M. Yuan. *KVTuner: Sensitivity-Aware Layer-Wise Mixed-Precision KV Cache Quantization for Efficient and Nearly Lossless LLM Inference*. ICML 2025, arXiv:2502.04420. Code: https://github.com/cmd2001/KVTuner.

[16] J. Zhang et al. *More Tokens, Lower Precision: Towards the Optimal Token-Precision Trade-off in KV Cache Compression*. arXiv:2412.12706, 2024.

[17] ggml-org, `llama.cpp` repository: KV-cache quantization types (`-cache-type-k/v`: fp16, q8_0, q4_0). https://github.com/ggml-org/llama.cpp.

[18] J. Kübler, E. Kurtić, et al. (vLLM team), *The State of FP8 KV-Cache and Attention Quantization in vLLM*, vLLM blog, 2026-04-22. https://vllm-project.github.io/2026/04/22/fp8-kvcache.html.

[19] Strata, `docs/DETAILS.md`: KV-cache quantization section (byte model, KL measurements, window 256/64 at 128K on both 128K texts). In-repo.

[20] Strata, `docs/QUANTS.md`: quant-quality yardsticks (mean KL, KL p99, top-1, perplexity) for the weight-quantization experiment; the cwe KL protocol is documented in the research-branch harness. In-repo.

[21] Strata, `bench/results/2026-09-27-kv-q4`: Q4_0 KV vs fp16/int8 document perplexity at 1K/8K (Q2_0). In-repo.

[22] Strata, `docs/KV_QUANT_WORKPACKAGE.md` (branch `KV_QUANT_research`): workpackage report, window history §7.12–§7.21, K-versus-V falsification, crossover withdrawal. In-repo.

[23] Strata, `bench/results/2026-10-24-needle-window`: same-day needle pair, engine 0.1.47, `int8` against the 256/64 window, 6/6 and 6/6. In-repo.